

YARA rule authoring and generalisation

Follow-along: write a detection rule against files you can read,
then find out what it does to files you cannot

Overview

A detection rule is a written description of a file, precise enough that a scanner can decide whether a file in front of it fits. **YARA** is the language such rules are usually written in, and it is used the same way by the people who publish signatures and by the people who write them for one organisation and never share them. This lab hands you a corpus of thirty Windows executables, eight of which are builds of one implant family, and asks you to write a rule that matches those eight and none of the other twenty-two.

Getting that right is not the exercise. The exercise is what happens next: the same rule is run against ten files you cannot open, built from the same family and from software you have never seen. Three obvious rules reach a perfect score on the corpus, and each of them matches a benign file in the holdout, because each is keyed on one feature that some benign program has for its own reasons. What separates a rule that works from a rule that generalises is which features it combines, and the only way to find that out is to be scored on files you could not tune against.

This lab stops where the samples would have to run. Everything here is read off a file at rest: sections, imports, strings and hashes. Nothing is executed, no debugger is opened, and no behaviour is observed, because none of that is possible for a Windows binary on a Linux host. What a rule built this way cannot see is anything the program only does once it is running.

Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab    # first time
cd ~/MiniLab && git pull                                       # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

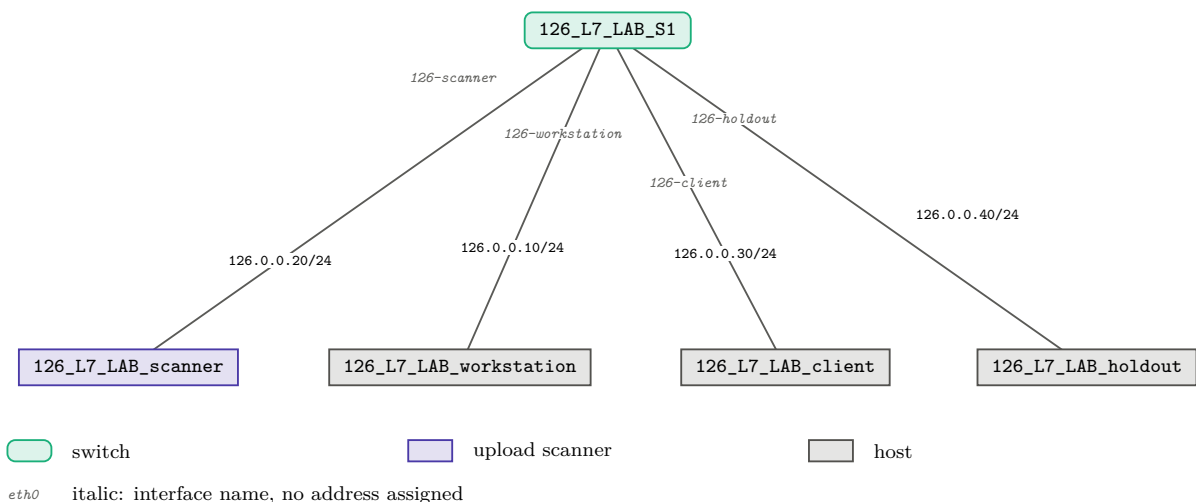
If it exits with a Docker error, follow the setup in the repository's **README.md** and run it again.

The root menu lists one category per row. Select **Malware analysis and detection engineering**, then **YARA rule authoring and generalisation**, to reach this lab's actions:

- **Spawn** creates this lab's containers and wiring. Wait for the panel to turn green before you go on. The first spawn of a lab also builds its images, which takes a few minutes; later spawns take seconds.
- **Status** prints the lab's state and its success oracle: whether your rule file compiles, how many files of each kind it matched, and what the upload scanner did with the two files the client sends it. Status takes over the whole screen: the arrow keys scroll its output, and Enter on **Done** closes it.
- **Shell command for a node** lists every node (workstation, scanner, client) and the `docker exec -it ...` line that opens a shell on it. Copy the line for the node a step below names into your own terminal. It holds the screen until Enter on **Done**, as Status does.
- **Reset to baseline** deletes your rule file and whatever you put on the scanner, and returns the lab to its clean Part 1 state without tearing it down.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Spawn the lab now, then work through Part 1 in your own terminal. **Back** (or Esc) returns to the lab list.

The setup



Four machines on one segment behind one switch. Everything in Parts 1 to 3 is done on the workstation; the scanner and the client exist for Part 4.

Machine	Address	What it is
126_L7_LAB_workstation	126.0.0.10	The analysis box. It holds the corpus at <code>/srv/corpus</code> , the tools you read it with, and the rule file you write.
126_L7_LAB_scanner	126.0.0.20	An upload endpoint. It accepts a file over HTTP, scans it with <code>clamscan</code> against whatever rule database you have deployed, and replies <code>ACCEPTED</code> or <code>REJECTED</code> .
126_L7_LAB_client	126.0.0.30	Posts two files to the scanner in Part 4.
126_L7_LAB_holdout	126.0.0.40	Holds the ten files Part 3 scores you against. There is no shell command for it and nothing in this handout sends you there.
126_L7_LAB_S1	–	One Open vSwitch bridge, no VLANs, no spanning tree.

The corpus

Thirty files under `/srv/corpus`, in two directories:

Directory	Family builds	Benign files	Used by
<code>/srv/corpus/set-a</code>	5	15	Part 1
<code>/srv/corpus/set-b</code>	3	7	Part 2 onwards

A file named `nightjar-NN.exe` is a build of the implant family. A file named `bn-NN.exe` is benign. You are given those labels because what this lab grades is the rule, not the triage: working out which of a set of unknown files is malicious is a different exercise.

Every file is read-only and every one of them is compiled from C source inside the lab's own container image when the image is built. No sample is downloaded, no real malware is involved, and no executable is ever written to the machine you are sitting at. They are genuine PE files with genuine import tables and section layouts, so every property in this handout is a property of the exact build in front of you; and they are inert, because Windows code does not execute on a Linux host and nothing in the lab tries.

The tools on the workstation

Command	What it does
<code>yara</code>	Compiles a rule file and reports which rules matched which files.
<code>readpe</code>	Prints the structures inside a PE file: headers, sections, imports.
<code>pestr</code>	Prints the printable strings in a PE file with the offset each sits at.
<code>peldd</code>	Prints the DLL names a PE file depends on.
<code>pehash</code>	Prints hashes of a PE file, including the imphash.
<code>upx</code>	The packer three of the family's builds were compressed with.
<code>file</code>	Names the file format and counts the sections.

Assessment

There are 19 questions, placed where the step that produces the answer sits. Answer all of them in a separate document. The Status action's counts are the lab's own measurement of your rule; several questions ask you to record and explain them.

Part 0: reading a Windows executable

Before a rule can be written, the features it might be written against have to be visible. This part is the reading; nothing here produces a rule.

Definition: PE, and its sections

PE (Portable Executable) is the file format Windows uses for `.exe` and `.dll` files. A PE file is divided into **sections**, each a named run of bytes with its own address in memory and its own permissions. A compiler emits conventional names: `.text` holds executable code, `.data` holds writable initialised variables, `.rdata` holds read-only ones, `.rsrc` holds resources, and `.idata` holds the import table. The names are a convention and nothing enforces them, which is why a section called `UPX0` is worth noticing.

Open a shell on the workstation and look at one family build.

```
# 126_L7_LAB_workstation: the analysis box
cd /srv/corpus
file -b set-a/nightjar-01.exe
readpe -S set-a/nightjar-01.exe | grep 'Name:'
```

`file` reports PE32+ executable for MS Windows 5.02 (console), x86-64, 18 sections. Eighteen is what this compiler emits by default: nine sections the program needs and nine `.debug_*` sections carrying DWARF debugging information that the linker did not strip. Compare it with `set-a/nightjar-02.exe`, which is the same source built with the linker's `-s` flag, and with `set-a/bn-01.exe`, which is a benign program built the same way as `nightjar-01.exe`.

Question 1. Give the section count `file` reports for `set-a/nightjar-01.exe`, `set-a/nightjar-02.exe` and `set-a/bn-01.exe`. Using those three numbers, say why a rule whose condition included `pe.number_of_sections == 18` would be wrong in both directions.

Definition: Import table, and imphash

A PE file does not contain the operating system's code. It carries an **import table** naming each DLL it needs and, for each one, the functions it calls out of it; the loader resolves those names to addresses when the program starts. The table is therefore a list of what the program is able to do, readable without running it. The **imphash** is the MD5 of that list, taken over the library and function names in the order the file stores them, so two builds of one program that call the same functions in the same order share an imphash and a program that calls anything else does not.

```
# 126_L7_LAB_workstation
peldd set-a/nightjar-01.exe
readpe -i set-a/nightjar-01.exe | grep -E 'Name:'
peldd set-a/bn-10.exe          # a benign program that downloads a file
peldd set-a/bn-08.exe          # a benign program that base64-encodes a file
```

Question 2. List the DLLs `peldd` reports for `set-a/nightjar-01.exe`, `set-a/bn-10.exe` and `set-a/bn-08.exe`. Name the one library the family imports that `bn-10.exe` does not and the one it imports that `bn-08.exe` does not, and say what each of those two libraries lets a program do.

Now the strings.

Definition: Configuration block

A remote access implant needs to know where to report to. That address, and whatever else varies between deployments, is a **configuration block**: a run of bytes the program reads at startup, usually stored as an ordinary initialised variable or as a resource. It is the part of a sample that changes from one victim to the next while the code stays the same, which makes it the most specific thing in the file and the least durable.

```
# 126_L7_LAB_workstation
pestr -o set-a/nightjar-01.exe | grep NJCFG3
```

The reply names an offset and the string sitting at it:

```
# 126_L7_LAB_workstation prints this; it is output, not commands to type
0x2000 NJCFG3|c2=126.9.0.40:8443|id=WKS-01|k=7f2a1c
```

`pestr` takes a second flag, `-s`, which prints a section name beside each string. Run it, then check what it says against the section table.

```
# 126_L7_LAB_workstation
pestr -o -s set-a/nightjar-01.exe | grep NJCFG3
readpe -S set-a/nightjar-01.exe | grep -A5 'Name: *\.data'
```

Question 3. `pestr -s` names one section for that string and the `.data` entry of the section table implies another. Give both, give the **Virtual Address** and the **Pointer To Raw Data** of `.data`, and use those two fields to say which of the two is right and what `pestr` is doing wrong.

The other four family builds in `set-a` are the same program compiled with different optimisation settings, carrying different configuration blocks, and in one case keeping the block in a resource rather than in `.data`. Their import tables differ accordingly.

```
# 126_L7_LAB_workstation
for f in set-a/nightjar-0*.exe; do printf '%s ' "$f"; pehash "$f" | grep -i
    imphash; done
```

Question 4. Four of the five imphashes in `set-a` are equal and one is not. Name the build that differs, and explain the split from the import tables: give the three `KERNEL32` functions it calls that the other four do not, and say what it uses them for.

Part 1: a rule against the files in front of you

Definition: YARA rule

A YARA rule has a name, an optional **meta** block of free-form key/value pairs, an optional **strings** block declaring the byte sequences to look for, and a **condition** that is a boolean expression over those strings and over anything the loaded modules expose. A string identifier begins with `$`; in a condition, the bare identifier means "this string appears somewhere in the file". `yara RULES TARGET` prints one line per matching rule, giving the rule name and the path, and prints nothing for a file that matched no rule.

The configuration block you found in Part 0 is the most specific thing in `nightjar-01.exe`. Write a rule around it.

```
# 126_L7_LAB_workstation: write the rule file the lab scores
cat > /root/rules.yar <<'EOF'
rule Nightjar_Config_Block
{
    meta:
        family = "Nightjar"

    strings:
        $cfg = "NJCFG3|c2=126.9.0.40:8443|id=WKS-01|k=7f2a1c" ascii

    condition:
        $cfg
}
EOF
```

`/root/rules.yar` is the file the Status action reads. Everything the lab scores comes out of that path, so keep working in it rather than in a copy.

```
# 126_L7_LAB_workstation: -r walks a directory
yara -r /root/rules.yar /srv/corpus/set-a
```

One warning about the command line before you go further. `yara` takes any number of *rule* files before the target, so `yara r.yar a.exe b.exe` tries to parse `a.exe` as a second rule file and stops with a syntax error. Scan one target at a time, or give it a directory with `-r`.

Question 5. Give the number of files in `set-a` this rule matches, and the number of family builds it misses. For one named build it misses, run `pestr -o` on it and quote the configuration block it carries instead, then say in one sentence what property of a configuration block makes a rule written around a whole one specific to a single build.

The part of the block that does not change is its first ten bytes, `NJCFG3|c2=`. Everything after that is the C2 address, the build tag and the key.

Question 6. Before you edit anything: predict how many of the twenty files in `set-a` a rule whose only string is `"NJCFG3|c2="` will match, and how many of those matches will be benign files. Then make the change, re-run the scan, and give the two numbers you actually got.

Part 2: the builds that are compressed

Definition: Runtime packer

A **packer** compresses a program's code and data into one blob and prepends a small stub. The file on disk holds the compressed blob; the stub decompresses it into memory at startup and jumps to the original entry point. UPX is the most common one and is an ordinary open-source compression tool rather than a malware technique, so a packed file is not by itself a malicious file. What packing changes for a rule writer is that the bytes a rule was written against are no longer in the file in the order the compiler put them.

`set-b` holds three more builds of the family and seven more benign files. Scan the whole corpus.

```
# 126_L7_LAB_workstation
yara -r /root/rules.yar /srv/corpus
file -b set-b/nightjar-06.exe
readpe -S set-b/nightjar-06.exe | grep 'Name:'
```

Three sections named UPX0, UPX1 and UPX2, where the unpacked build had eighteen. Take a copy and reverse it, because a rule writer wants to know what the compressor kept and what it did not.

```
# 126_L7_LAB_workstation: /srv/corpus is read-only, so work on a copy
cp set-b/nightjar-06.exe /root/work/n06.exe
chmod 644 /root/work/n06.exe
upx -d /root/work/n06.exe
pestr -o /root/work/n06.exe | grep NJCFG3
pestr -o set-b/nightjar-06.exe | grep NJCFG3
```

Question 7. Quote the string `pestr` finds in the decompressed copy and the string it finds in the packed file, both in full. Give the number of leading bytes that survived compression, and explain the difference between them: say what a compressor emits for the first occurrence of a run of bytes it has never seen before, and what it emits once it has seen that run.

The import table is the other half of what packing does or does not preserve.

```
# 126_L7_LAB_workstation
peldd set-b/nightjar-06.exe
readpe -i set-b/nightjar-06.exe | grep 'Name:'
pehash set-b/nightjar-06.exe | grep -i imphash
```

Question 8. The packed build's import table lists four DLLs, as the unpacked build does, but a different number of function names. Give the function names it keeps for `WININET.dll` and for `CRYPT32.dll`, say why the stub needs the DLL to appear in the table at all, and name the two `KERNEL32` functions in that table the stub uses to find everything else.

Question 9. Give the imphash of `set-b/nightjar-06.exe` and of `set-a/nightjar-01.exe`, which are the same source built with the same flags and then compressed. Using that pair and your answer to Question 4, state the two separate reasons an imphash rule would fail to cover this family.

Definition: Module, and the `pe` module

A YARA **module** exposes structured facts about a file to the condition, beyond what a byte string can express. `import "pe"` at the top of a rule file loads the PE module, after which a condition can use `pe.is_pe`, `pe.number_of_sections`, `pe.sections[i].name`, and `pe.imports(dll)` or `pe.imports(dll, function)`, which are true when the file's import table names that library, or that library and that function. The library name is matched without regard to case.

Three features separate the family from the benign files in this corpus, and the module gives you two of them. Put all three in the file as three rules, so that `yara` names which of them matched each file.

```
# 126_L7_LAB_workstation
cat > /root/rules.yar <<'EOF'
import "pe"

rule Nightjar_Tag_Only
{
    strings:
        $tag = "NJCFG3|c2=" ascii
    condition:
        $tag
}

rule Nightjar_Import_Pair_Only
{
    condition:
        pe.is_pe and pe.imports("wininet.dll") and pe.imports("crypt32.dll")
}

rule Nightjar_Upx_Sections_Only
{
    condition:
        for any i in (0 .. pe.number_of_sections - 1) : (
            pe.sections[i].name == "UPX0"
        )
}
EOF
yara -r /root/rules.yar /srv/corpus
```

Question 10. For each of the three rules, give the number of family builds and the number of benign files it matched across the whole corpus. One of the three matches benign files; name them and say, from `peldd` and `readpe -S`, what each of them has that made the rule fire.

Question 11. Two of the three rules are perfect on this corpus: every family build, no benign file. Explain why that result on its own is not evidence that either rule would be perfect on a file outside the corpus. Your answer should name what a corpus of twenty-two benign files is a sample of.

Part 3: the files you cannot read

Definition: Holdout

A **holdout** is a set of labelled files kept back from whoever writes the rule, scored against only once the rule is finished. Its purpose is that a rule can be tuned against anything its author can read, so a score on the corpus measures how well the rule fits that corpus and nothing else. A score on files the author never saw measures whether the rule describes the family or the sample.

The holdout container holds ten files: three further builds of the family and seven benign programs. Their configuration blocks, their compiler settings and the software they were built from appear nowhere in `/srv/corpus`. Run the Status action.

Status copies `/root/rules.yar` into the holdout container, runs `yara` there, and prints two numbers: how many of the three family builds matched, and how many of the seven benign

files matched. It never prints a filename, so the counts are all you get and you cannot work out which file caused a false positive by elimination. You may run it as often as you like.

Question 12. Before running Status: for each of the two rules that were perfect on the corpus, describe the benign program that would defeat it. Say what that program would have to import, or contain, and give a plausible reason for a legitimate program to be built that way.

Question 13. Run Status and record the four counts for the rule file as it stands. State which of your three rules is responsible for each false positive, and how you can tell that from the counts alone.

Both of the corpus-perfect rules have a false positive here, and they do not have the same one. That is what makes the conjunction of the two stronger than either: a benign file that imports both libraries does not carry the family's tag, and a benign file that carries the tag imports neither library.

```
# 126_L7_LAB_workstation: one rule, both features required
cat > /root/rules.yar <<'EOF'
import "pe"

rule Nightjar_Implant
{
    meta:
        family      = "Nightjar"
        confidence = "high"

    strings:
        $tag = "NJCFG3|c2=" ascii

    condition:
        pe.is_pe
        and pe.imports("wininet.dll")
        and pe.imports("crypt32.dll")
        and $tag
}
EOF
```

Run Status again. The mark for Parts 1 to 3 is eight of eight on the corpus and three of three on the holdout with no false positive in either, and Status prints **RULE PASS** when your file reaches it.

Question 14. Take the rule apart and say which conjunct is doing the work in each case: name the conjunct that excludes the holdout's benign file that imports both libraries, and the conjunct that excludes the holdout's benign file that carries the tag. Then say which conjunct excludes neither of them, and why it is in the rule anyway.

Question 15. Describe a build of this family that your rule would miss. It has to be a change the family's authors could make without giving up what the program does, so name the change, name the conjunct it defeats, and say what you would have to look at instead.

Part 4: deploying the rule

A rule that scores well in an analyst's terminal has not stopped anything yet. Part 4 puts it in front of traffic.

The scanner at 126.0.0.20 accepts a file over HTTP and scans it before deciding what to do with it. It runs `clamscan` rather than `yara`, because ClamAV is what the endpoint was already running and it reads a `.yar` file as a signature database directly. The database it consults is `/etc/minilabs/deployed.yar`, and at spawn there is no such file.

Definition: Signature database

A scanner does not hold its rules in its program text. It loads them from a **signature database**: one or more files named on the command line or found in a configured directory, parsed at startup. Which rule languages a given scanner accepts, and which parts of a language it implements, is a property of that scanner rather than of the language, so a rule file that one tool compiles is not guaranteed to load in another.

Deploy what you have.

```
# 126_L7_LAB_workstation: read the rule file back out
cat /root/rules.yar
```

```
# 126_L7_LAB_scanner: paste the body of your own rule where the marker is,
# then run the whole block
cat > /etc/minilabs/deployed.yar <<'EOF'
PASTE THE CONTENTS OF /root/rules.yar HERE
EOF
```

Now send it a file.

```
# 126_L7_LAB_client: post a packed build of the family to the scanner
cd /root/uploads
curl -s --data-binary @invoice-viewer.exe \
    'http://126.0.0.20/cgi-bin/upload.cgi?name=invoice-viewer.exe'
```

Read the whole reply, not just its last line. The scanner copies everything `clamscan` writes to its standard error into the reply, above the verdict.

Question 16. Give the verdict the scanner returned for `invoice-viewer.exe`, and quote the two LibClamAV lines above it. Say how many rules `clamscan` loaded, what it therefore compared the file against, and what its exit status was. Then say why a scanner that behaves this way is worse than one that refused to start.

Question 17. Scan the same kind of file with `yara` on the workstation (`yara /root/rules.yar /srv/corpus/set-b/nightjar-06.exe`) and state the result. Using the two results together, say whether the problem is in the rule or in the tool reading it, and name the specific construct in your rule that `clamscan` rejected.

The family's two libraries have to be expressed as something `clamscan` does support. An imported function's name is not metadata sitting outside the file: the import table stores it as a NUL-terminated string in the file's bytes, which is how `readpe -i` read it back in Part 0. A literal string is therefore available where `pe.imports()` is not.

Write a second rule using only strings and a condition over them, deploy it, and send both of the client's files.

```
# 126_L7_LAB_client: the second file is a benign triage helper
cd /root/uploads
curl -s --data-binary @invoice-viewer.exe \
    'http://126.0.0.20/cgi-bin/upload.cgi?name=invoice-viewer.exe'
curl -s --data-binary @logtag-check.exe \
    'http://126.0.0.20/cgi-bin/upload.cgi?name=logtag-check.exe'
```

The mark for Part 4 is both verdicts at once: `invoice-viewer.exe` rejected and `logtag-check.exe` accepted. Status prints `SCANNER PASS` when it reaches that, and runs both uploads for you.

Question 18. Run `pestr -o logtag-check.exe` on the client and quote the family tag it holds. Explain why that file is benign despite carrying it, and say what a database whose only string is the tag would do to this upload.

Question 19. Give the strings your deployed rule requires and the condition over them. Then compare one of them against the module expression it replaced: describe a file that `pe.imports("wininet.dll")` would call a non-match and a string match on `"InternetOpenA"` would call a match, and say which of the two verdicts on that file is the one a detection engineer wants.

Checks: what "done" means

Run the Status action. It prints two verdicts and both have to be reached:

- **RULE PASS.** Your `/root/rules.yar` matches all eight family builds in `/srv/corpus` and none of the twenty-two benign files, and matches all three family builds in the holdout and none of its seven benign files.
- **SCANNER PASS.** The database at `/etc/minilabs/deployed.yar` makes the scanner reject `invoice-viewer.exe` and accept `logtag-check.exe`.

Status also prints the name of every corpus file your rule matched wrongly, which is how you find a false positive there. For the holdout it prints counts only.

If you want to start over, the Reset to baseline action deletes your rule file and the scanner's database and leaves the corpus and the holdout untouched.

Safety note

Every machine in this lab sits on one Open vSwitch bridge with no route off the host, so the two uploads in Part 4 go from one container to another and nowhere else.

None of the thirty corpus files, the ten holdout files or the two uploads is real malware. Each is compiled from C source held in the lab's own image, inside the image build, so no executable that imports `WININET` and is compressed with `UPX` is ever written to the filesystem of the machine you are working on. They are Windows binaries on a Linux host and cannot be executed here, and no script in this lab tries.

Tear the lab down

When you have finished, run the **Teardown** action, or `scripts/teardown.sh` from the lab directory. It removes this lab's five containers and nothing else.